WEB TECHNOLOGIES — MODULE 3 & 4

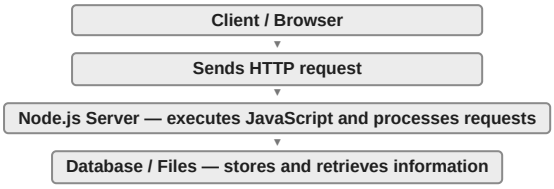
20CS212 · CHEATSHEET NOTES · AJAY.I – 71812401007

MODULE 3 — SERVER-SIDE PROGRAMMING (Node.js)

1. Node.js Introduction

Definition: Node.js is an open-source, cross-platform JavaScript runtime environment that allows JavaScript to execute outside the browser.

Normally, JavaScript runs inside a browser. Node.js allows it to run on a server or computer.



Server sends the processed response back to the browser.

Important features of Node.js

1. Asynchronous: It can start operations without waiting for each operation to finish.
2. Event-driven: Executes event handlers when events occur.
3. Non-blocking I/O: I/O operations need not block the execution of other work.
4. Single main JavaScript thread: Uses an event loop to coordinate asynchronous operations.
5. Cross-platform: Runs on Windows, Linux and macOS.
6. NPM: Provides access to a large collection of reusable packages.
7. Fast execution: Uses Google's V8 JavaScript engine.

Advantages:

- Suitable for real-time applications.
- Handles many concurrent connections efficiently.
- Uses JavaScript on both client and server.
- Supports scalable network applications.

PYQ preparation: "List the key features of Node.js" – 3 marks.

2. Node.js Modules

A module is a reusable block of code that performs a specific function.

Instead of writing an entire application in one file, Node.js allows us to divide it into smaller files and reuse them.

There are three important categories.

1. Built-in modules

Provided by Node.js itself.

Module	Purpose
`fs`	File operations
`http`	Create HTTP servers
`path`	Work with file paths
`os`	Operating system information
`events`	Event handling
`url`	URL operations

2. Core third-party modules

Packages installed through NPM.

Examples: `express`, `mongoose`.

3. User-defined modules

Modules created by the developer.

Example:

File: `math.js`

```javascript
function add(a, b) {
    return a + b;
}

module.exports = add;
```

File: `app.js`

```javascript
const add = require('./math');

console.log(add(10, 20));
```

Output:

```plain text
30
```

Explanation:

- `module.exports` exports a function from a file.
- `require()` imports the exported function.
- The function can now be reused in another file.

Important exam question: Explain Node.js modules with an example.

3. Callback Functions in Node.js

7-mark PYQ

A callback is a function passed as an argument to another function. It is called after a particular operation is completed or when a specific condition occurs.

Callbacks are especially useful in asynchronous programming.

Why callbacks are needed

Imagine reading a file that takes two seconds.

Without asynchronous execution, the program may have to wait before performing other operations.

With asynchronous execution, Node.js can continue other work while the file is being read. The callback executes when the operation finishes.

Example – Reading a file using callback

```javascript
const fs = require('fs');

console.log("Start");

fs.readFile("data.txt", "utf8",
    function(err, data) {

        if (err) {
            console.log(err);
            return;
        }

        console.log(data);
    }
);

console.log("End");
```

Output, assuming the read finishes after the immediate log statements:

```plain text
Start
End
Hello World
```

Working:

1. Import the File System module.
2. `readFile()` starts an asynchronous file-reading operation.
3. The callback is registered to execute after the operation completes.
4. Node.js continues executing the next statement.
5. When file reading completes, the callback receives the error or data.
6. The content is displayed.

Advantages:

- Supports asynchronous operations.
- Prevents unnecessary waiting.
- Improves application responsiveness.

Important: In Node.js, callbacks commonly use the error-first convention: the first argument represents an error and the second contains the result.

4. Events in Node.js – EventEmitter Pattern

Important PYQ – 7 marks

Node.js uses an event-driven architecture.

An event is an action or occurrence that the application can detect and respond to.

Examples:

- File opened.
- User connected.
- Data received.
- Server request received.

Node.js provides the built-in `events` module to create and handle custom events.

Important EventEmitter methods

Method	Purpose
`on()`	Registers an event listener
`emit()`	Triggers an event
`once()`	Executes listener only once
`removeListener()`	Removes a particular listener
`removeAllListeners()`	Removes listeners for an event

Master program – EventEmitter

```javascript
const EventEmitter = require('events');

// Create emitter object
const emitter = new EventEmitter();

// Create listener
emitter.on('message', function() {
    console.log("Message received!");
});

// Trigger event
emitter.emit('message');
```

Output:

```plain text
Message received!
```

Working:

1. Import the `events` module.
2. Create an EventEmitter object.
3. Register a listener using `on()`.
4. Trigger the event using `emit()`.
5. The registered callback function executes.

Another example – Event with data

```javascript
const EventEmitter = require('events');

const emitter = new EventEmitter();

emitter.on('student', function(name) {
    console.log("Welcome " + name);
});

emitter.emit('student', "Ajay");
```

Output:

```
Welcome Ajay
```

Difference between callbacks and events

Callback	Event
Function passed to another function	An occurrence that can be emitted
Usually associated with a particular operation	Can have multiple registered listeners
Invoked by the function performing the operation	Triggered using `emit()`
Example: `fs.readFile()`	Example: `EventEmitter`

Remember: Callback = function passed to a function. Event = occurrence detected and handled by listeners.

5. NPM – Node Package Manager

3-mark PYQ + installation steps

NPM stands for Node Package Manager.

It is the default package manager distributed with Node.js. It is used to install, manage and share JavaScript packages.

Why do we need NPM?

Suppose we want to build a web server. Instead of implementing every feature ourselves, we can install an existing package such as Express.

Installation and usage

First install Node.js from its official website. NPM is included with the installation.

Check installation:

```bash
node -v
npm -v
```

Create a project:

```bash
mkdir myproject
cd myproject
```

Initialise the project:

```bash
npm init -y
```

Install a package:

```bash
npm install express
```

Install a development dependency:

```bash
npm install nodemon --save-dev
```

Important NPM commands

Command	Purpose
`npm -v`	Check NPM version
`npm init`	Create package.json
`npm init -y`	Initialise with default values
`npm install express`	Install package
`npm uninstall express`	Remove package
`npm update`	Update packages
`npm list`	Display installed packages
`npm start`	Run the configured start script

Important files

- `package.json` – Contains project details, scripts and dependencies.
- `package-lock.json` – Records the dependency versions for reproducible installation.
- `node_modules` – Stores installed packages.

6. Node.js File System (FS)

3-mark PYQ + 2-mark PYQ

The File System module allows Node.js to create, read, update, append and delete files.

It is imported using:

```javascript
const fs = require('fs');
```

Important file operations

Method	Purpose
`writeFile()`	Creates or overwrites a file
`readFile()`	Reads a file
`appendFile()`	Adds content to a file
`rename()`	Renames a file
`unlink()`	Deletes a file

Master program – File operations

```javascript
const fs = require('fs');

// Create a file
fs.writeFile('data.txt', 'Hello World',
    function(err) {
        if (err) throw err;

        console.log("File created");

        // Read file
        fs.readFile('data.txt', 'utf8',
            function(err, data) {
                if (err) throw err;

                console.log(data);

                // Append data
                fs.appendFile('data.txt', '\nWelcome',
                    function(err) {
                        if (err) throw err;

                        console.log("Data added");

                        // Read updated file
                        fs.readFile('data.txt', 'utf8',
                            function(err, updated) {
                                if (err) throw err;

                                console.log(updated);

                                // Delete file
                                fs.unlink('data.txt',
                                    function(err) {
                                        if (err) throw err;

                                        console.log("File deleted");
                                    });
                            });
                    });
            });
    });
```

Expected output:

```
File created
Hello World
Data added
Hello World
Welcome
File deleted
```

Explanation:

1. `writeFile()` creates the file.
2. `readFile()` reads the content.
3. `appendFile()` adds new content.
4. `readFile()` reads the updated content.
5. `unlink()` deletes the file.

Important exam point: The operations are nested to ensure that each subsequent operation happens after the previous one completes.

7. Buffers in Node.js

A Buffer is a built-in Node.js object used to handle raw binary data.

Computers store data in binary form. When Node.js receives files, images or network data, it may need to process the raw bytes.

Buffers allow Node.js to store and manipulate these bytes directly.

Applications:

- Reading images.
- Processing audio and video.
- Handling network packets.
- Working with binary files.

Creating a Buffer

```javascript
const buf = Buffer.from("Hello");

console.log(buf);
console.log(buf.toString());
console.log(buf.length);
```

Output:

```
<Buffer 48 65 6c 6c 6f>
Hello
5
```

Explanation:

- `Buffer.from()` creates a buffer from existing data.
- `toString()` converts buffer content into text.
- `length` gives the number of bytes.

Other methods

```javascript
Buffer.alloc(10);        // Creates 10-byte buffer

Buffer.from("ABC");      // Creates from string

buf.toString();          // Converts to string

buf.length;              // Buffer size
```

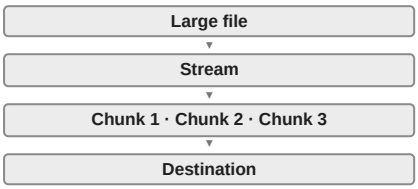
Important distinction: A JavaScript string represents text; a Buffer represents bytes.

8. Streams in Node.js

A stream is a mechanism used to process data continuously in smaller chunks instead of loading the entire data into memory at once.

Consider reading a 2 GB video file.

- Without streams, the entire file may need to be loaded into memory.
- With streams, data can be read and processed in smaller chunks.



Four types of streams

Type	Purpose
Readable	Reads data
Writable	Writes data
Duplex	Reads and writes data
Transform	Reads data, modifies it and outputs the result

Master program – Readable and writable stream

```javascript
const fs = require('fs');

const readStream =
    fs.createReadStream('input.txt');

const writeStream =
    fs.createWriteStream('output.txt');

readStream.on('data', function(chunk) {

    console.log("Chunk received");

    writeStream.write(chunk);

});

readStream.on('end', function() {

    console.log("Reading completed");

    writeStream.end();

});
```

Working:

1. `createReadStream()` creates a readable stream.
2. `createWriteStream()` creates a writable stream.
3. The `data` event is triggered when a chunk arrives.
4. `write()` writes the received chunk.
5. The `end` event indicates that reading is complete.
6. `end()` finishes the writable stream.

Advantages:

- Efficient memory usage.
- Suitable for large files.
- Processes data incrementally.
- Useful in video streaming and network communication.

PYQ-style question: Differentiate between Buffer and Stream.

Buffer	Stream
Stores binary data in memory	Processes data in chunks
Usually handles a complete available data block	Handles data incrementally
Useful for small or already-loaded data	Useful for large data
Example: `Buffer.from()`	Example: `createReadStream()`

9. Listener and Emitter Pattern

This pattern is closely connected to EventEmitter.

An emitter generates an event, while a listener waits for that event and executes a function when it occurs.

Example:

```javascript
const EventEmitter = require('events');

const emitter = new EventEmitter();

// Listener
emitter.on('login', function(user) {
    console.log(user + " logged in");
});

// Emitter
emitter.emit('login', "Ajay");
```

Output:

```
Ajay logged in
```

Real-world example – Notification system

- User places an order.
- Order system emits an `orderPlaced` event.
- Notification system listens for that event.
- Notification is sent to the user.

This separates the event producer from the event handler.

Difference between callback, listener and emitter:

- Callback: Function passed to another function.
- Listener: Function registered to respond to an event.
- Emitter: Object that triggers the event.

MODULE 3 — REACT JS

10. Introduction to ReactJS

ReactJS is an open-source JavaScript library used to build interactive user interfaces, especially single-page applications.

It was developed by Facebook (now Meta).

React allows developers to divide a webpage into small reusable components.

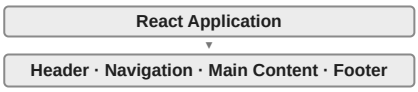
For example, an online shopping application can have:

- Header component.
- Product component.
- Shopping cart component.
- Login component.
- Footer component.

Important features of ReactJS

1. Component-based architecture.
2. Reusable UI components.
3. Virtual DOM for efficient UI updates.
4. JSX support.
5. One-way data flow through props.
6. State management.
7. Event handling.

React application architecture



Each section can be implemented as a separate reusable component.

11. ReactJS Toolchain

The React toolchain refers to the software tools required to develop, run and build React applications.

Important tools

Tool	Purpose
Node.js	Provides JavaScript runtime
NPM	Installs and manages packages
React	Builds user interfaces
JSX	Allows HTML-like syntax in JavaScript
Vite	Development server and build tool
Babel	Transforms JSX and modern JavaScript when configured
Browser	Displays the React application

Creating a React application using Vite

```bash
npm create vite@latest myapp -- --template react

cd myapp

npm install

npm run dev
```

The development server displays a local URL, usually:

```
http://localhost:5173
```

Important files

File	Purpose
`package.json`	Dependencies and scripts
`index.html`	Main HTML entry point
`src/main.jsx`	React application entry point
`src/App.jsx`	Main application component
`src/App.css`	Application styling

Important distinction: Vite is a development/build tool. React is the library used to create UI components.

12. Render Element in React

Rendering means displaying React elements and components in the browser.
React uses a root to connect the application to an HTML element.

Example

HTML:

```html
<div id="root"></div>
```

JavaScript:

```jsx
import React from 'react';
import { createRoot } from 'react-dom/client';

const element = <h1>Hello World</h1>;

const root = createRoot(
    document.getElementById('root')
);

root.render(element);
```

Output (illustrative browser output):

```
Hello World
```

Working:

1. `document.getElementById()` selects the root HTML element.
2. `createRoot()` creates a React root.
3. `root.render()` displays the React element.
4. The browser displays Hello World.

In modern React, `createRoot()` is the standard entry point. Older examples may use `ReactDOM.render()`.

13. ES6 Class Definition

ES6 introduced the `class` keyword to create objects using class-based syntax.
A class acts as a blueprint for creating objects.

Syntax

```javascript
class Student {

    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    display() {
        console.log(this.name);
        console.log(this.age);
    }

}

const s1 = new Student("Ajay", 20);

s1.display();
```

Output:

```
Ajay
20
```

Important terms:

- `class` – Defines a class.
- `constructor()` – Initialises object properties.
- `this` – Refers to the current instance.
- `new` – Creates an object.
- Methods – Functions defined inside the class.

React class component

A class component is a React component created using an ES6 class that extends `React.Component`.

```jsx
import React from 'react';

class Welcome extends React.Component {

    render() {
        return <h1>Hello Ajay</h1>;
    }

}

export default Welcome;
```

The `render()` method returns the JSX that describes the component's UI.

14. JSX in ReactJS

3-mark PYQ

JSX stands for JavaScript XML.

It is a syntax extension that allows developers to write HTML-like code inside JavaScript.

Without JSX

```javascript
const element = React.createElement(
    'h1',
    null,
    'Hello World'
);
```

With JSX

```jsx
const element = <h1>Hello World</h1>;
```

JSX is transformed into JavaScript calls by the React build toolchain.

Important JSX rules

1. Return a single parent element, or use a fragment.
2. Close all tags.
3. Use `className` instead of `class`.
4. Use `{}` to insert JavaScript expressions.
5. Use camelCase for many event attributes, such as `onClick`.

Example:

```jsx
function Student() {

    let name = "Ajay";

    return (
        <div>
            <h2>{name}</h2>
            <p>Computer Science</p>
        </div>
    );

}
```

Output:

```
Ajay
Computer Science
```

15. React Components

A component is an independent and reusable part of a React user interface.
Components help divide large applications into smaller manageable parts.

Functional component

```jsx
function Welcome() {
    return <h1>Welcome to React</h1>;
}
```

Using the component

```jsx
function App() {
    return (
        <div>
            <Welcome />
            <Welcome />
        </div>
    );
}
```

The same component can be reused multiple times.

Types of components in React

- Functional components.
- Class components.

Functional components are commonly used in modern React development.

16. React Props

Important 2-mark + 10-mark concept

Props stands for properties.

Props are used to pass data from a parent component to a child component.

They allow components to receive information and display different content.

Example

```jsx
function Student(props) {

    return (
        <h2>Hello {props.name}</h2>
    );

}

function App() {

    return (
        <div>
            <Student name="Ajay" />
            <Student name="Arun" />
        </div>
    );

}
```

Output:

```
Hello Ajay
Hello Arun
```

Working:

1. App is the parent component.
2. Student is the child component.
3. The parent sends `name` through props.
4. The child accesses it using `props.name`.
5. The child displays the received value.

Important properties of props:

- Passed from parent to child.
- Read-only from the receiving component's perspective.
- Allow component reuse.
- Can pass strings, numbers, objects and functions.

17. Component State

Very important

State is an internal data storage mechanism used by a React component.

Unlike props, state can be updated by the component, and changing state causes React to update the UI when necessary.

Example – Counter application

```jsx
import { useState } from 'react';

function Counter() {

    const [count, setCount] = useState(0);

    return (
        <div>
            <h2>Count: {count}</h2>

            <button onClick={() =>
                setCount(count + 1)
            }>
                Increase
            </button>

        </div>
    );

}

export default Counter;
```

Output initially:

```
Count: 0
```

After clicking Increase:

```
Count: 1
```

Explanation:

- `useState(0)` creates a state variable with initial value 0.
- `count` stores the current state.
- `setCount()` updates the state.
- React re-renders the component when the state changes.

Props vs State

Props	State
Passed from parent	Managed inside component
Read-only to child	Updated using setter
Used to receive data	Used to maintain changing data
Changes when parent supplies new values	Changes through state updates

18. Input Handling in React

Input handling allows users to enter data into React applications.

React commonly uses controlled components, where the input value is controlled through state.

Example:

```jsx
import { useState } from 'react';

function App() {

    const [name, setName] = useState("");

    return (
        <div>

            <input
                type="text"
                value={name}
                onChange={(e) =>
                    setName(e.target.value)
                }
            />

            <h2>Hello {name}</h2>

        </div>
    );

}

export default App;
```

Working:

1. `useState()` creates a name variable.
2. `value={name}` connects the input to state.
3. `onChange` detects changes in the input.
4. `e.target.value` gets the entered value.
5. `setName()` updates the state.
6. React displays the latest name.

19. Event Handling in React

Events allow React applications to respond to user actions.

Examples:

- Click.
- Change.
- Submit.
- Mouse over.
- Keyboard input.

React uses camelCase event names.

Event	Purpose
`onClick`	Button click
`onChange`	Input change
`onSubmit`	Form submission
`onMouseOver`	Mouse over
`onKeyDown`	Key pressed

Example:

```jsx
function App() {

    function greet() {
        alert("Welcome to React");
    }

    return (
        <button onClick={greet}>
            Click Me
        </button>
    );

}
```

Important: Write `onClick={greet}`, not `onClick={greet()}` when you want the function to execute in response to the click.

20. React Lifecycle

Left-out syllabus topic

The lifecycle describes the different stages through which a React component passes during its existence.

For class components, the lifecycle is divided into three major phases.

1. Mounting

Mounting is the process of creating a component and inserting it into the DOM.

Important methods:

- `constructor()`
- `render()`
- `componentDidMount()`

2. Updating

Updating occurs when the component's props or state change.

Important methods:

- `render()`
- `componentDidUpdate()`

3. Unmounting

Unmounting occurs when a component is removed from the DOM.

Important method:

- `componentWillUnmount()`

```
MOUNTING     constructor → render → componentDidMount
UPDATING     render → componentDidUpdate
UNMOUNTING   componentWillUnmount
```

Example

```jsx
class Student extends React.Component {

    componentDidMount() {
        console.log("Component mounted");
    }

    componentDidUpdate() {
        console.log("Component updated");
    }

    componentWillUnmount() {
        console.log("Component removed");
    }

    render() {
        return <h2>Hello Student</h2>;
    }

}
```

Applications:

- Fetching initial data.

- Setting up event listeners.
- Updating UI after state changes.
- Cleaning up timers and subscriptions.

Modern functional components use hooks such as `useEffect()` to handle comparable side effects.

21. React Forms

Forms are used to collect user information.

React forms usually use controlled inputs and state to manage user data.

Example:

```jsx
import { useState } from 'react';

function Registration() {

    const [name, setName] = useState("");
    const [email, setEmail] = useState("");

    function handleSubmit(e) {

        e.preventDefault();

        alert("Name: " + name +
              "\nEmail: " + email);

    }

    return (
        <form onSubmit={handleSubmit}>

            <input
                type="text"
                placeholder="Name"
                value={name}
                onChange={e =>
                    setName(e.target.value)
                }
                required
            />

            <input
                type="email"
                placeholder="Email"
                value={email}
                onChange={e =>
                    setEmail(e.target.value)
                }
                required
            />

            <button type="submit">
                Register
            </button>

        </form>
    );

}

export default Registration;
```

Important concepts:

- `value` connects input with state.
- `onChange` updates state.
- `onSubmit` handles form submission.
- `preventDefault()` prevents the browser's normal page reload on submission.
- `required` performs basic browser validation.

22. Styling with React

8-mark PYQ – May 2025

React supports different methods of applying CSS styles to components.

1. Inline styling

CSS is applied directly through a JavaScript object.

```jsx
function App() {

    const style = {
        color: "blue",
        fontSize: "24px",
        backgroundColor: "lightgray"
    };

    return (
        <h1 style={style}>
            Welcome to React
        </h1>
    );

}
```

Notice that CSS property names use camelCase, such as `fontSize`.

2. External CSS

Create `App.css`:

```css
.heading {
    color: blue;
    font-size: 24px;
    text-align: center;
}
```

React component:

```jsx
import './App.css';

function App() {

    return (
        <h1 className="heading">
            Welcome to React
        </h1>
    );

}
```

3. CSS Modules

CSS Modules allow styles to be locally scoped to components.

```css
/* App.module.css */

.heading {
    color: green;
}
```

```jsx
import styles from './App.module.css';

function App() {

    return (
        <h1 className={styles.heading}>
            Hello React
        </h1>
    );

}
```

4. CSS frameworks

React applications can also use CSS frameworks such as Bootstrap and Tailwind CSS.

Example:

```jsx
function App() {

    return (
        <button className="btn btn-primary">
            Submit
        </button>
    );

}
```

Advantages of styling in React:

- Improves user interface appearance.
- Allows component-level styling.
- Supports reusable styles.
- Helps create responsive applications.

23. Main 10-mark program – React application

Keep this as your master React program

Your uploaded notes use a Recipe Management application. It demonstrates most of the important React syllabus concepts in one program.

Instead of memorising separate programs, learn this one and understand how each part works.

Question: Develop a ReactJS application for managing recipes with options to add, delete, search and display recipes.

The program demonstrates:

- JSX.
- Components.
- Props.
- State.
- Input handling.
- Events.
- Forms.
- CSS styling.

```jsx
import React, { useState } from 'react';
import './App.css';

function Recipe({ recipe, onDelete }) {
    return (
        <div className="card">
            <h3>{recipe.name}</h3>
            <p>{recipe.description}</p>
            <button onClick={() => onDelete(recipe.id)}>
                Delete
            </button>
        </div>
    );
}

function App() {

    const [recipes, setRecipes] = useState([
        {
            id: 1,
            name: "Pizza",
            description: "Cheese Pizza"
        },
        {
            id: 2,
            name: "Pasta",
            description: "Creamy Pasta"
        }
    ]);

    const [name, setName] = useState("");
```

```
    const [description, setDescription] = useState("");
    const [search, setSearch] = useState("");

    function addRecipe(e) {
        e.preventDefault();

        if (!name.trim() || !description.trim()) return;

        const newRecipe = {
            id: Date.now(),
            name: name,
            description: description
        };

        setRecipes([...recipes, newRecipe]);

        setName("");
        setDescription("");
    }

    function deleteRecipe(id) {
        setRecipes(
            recipes.filter(recipe => recipe.id !== id)
        );
    }

    const filteredRecipes = recipes.filter(recipe =>
        recipe.name.toLowerCase()
        .includes(search.toLowerCase())
    );

    return (
        <div className="container">

            <h1>Recipe Management</h1>

            <form onSubmit={addRecipe}>

                <input
                    placeholder="Recipe Name"
                    value={name}
                    onChange={e => setName(e.target.value)}
                    required
                />

                <input
                    placeholder="Description"
                    value={description}
                    onChange={e =>
                        setDescription(e.target.value)
                    }
                    required
                />

                <button type="submit">
                    Add Recipe
                </button>

            </form>

            <input
                placeholder="Search Recipe"
                value={search}
                onChange={e => setSearch(e.target.value)}
            />

            {filteredRecipes.map(recipe => (
                <Recipe
                    key={recipe.id}
                    recipe={recipe}
                    onDelete={deleteRecipe}
                />
            ))}

        </div>
    );
}

export default App;
```

CSS – `App.css`

```
body {
    font-family: Arial;
    background: #f4f4f4;
}

.container {
    width: 80%;
    margin: auto;
    text-align: center;
}

input {
    padding: 10px;
    margin: 8px;
}

button {
    padding: 10px;
    background: green;
    color: white;
    border: none;
    cursor: pointer;
}

.card {
    background: white;
    padding: 15px;
    margin: 10px;
    border: 1px solid #ccc;
}
```

How the program works:

1. `useState()` stores the recipes, input fields and search value.
2. `Recipe` is a reusable child component.
3. Props pass the recipe information and delete function to the child.
4. `onChange` handles input changes.
5. `onSubmit` handles the form.
6. `addRecipe()` adds a new recipe.
7. `deleteRecipe()` removes a recipe.
8. `filter()` searches recipes.
9. `map()` displays the recipe components.
10. CSS styles the application.

How to adapt it to other questions

If the question asks for	Change the program to
Employee management	Employee name, position, salary
Student management	Student name, register number, department
Product management	Product name, price, quantity
Movie management	Movie title, genre, rating
Recipe management	Keep the original program

The core operations remain the same: state, inputs, props, events, add, delete and search.

MODULE 4 — CONNECTING WITH DATABASES (Node.js & NoSQL)

24. Introduction to NoSQL Database

NoSQL stands for Not Only SQL.

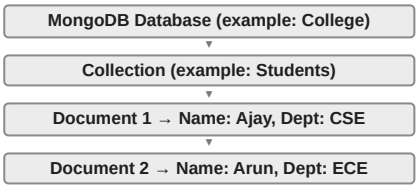
It is a type of database used to store and manage data without relying exclusively on traditional relational tables.

Unlike relational databases, NoSQL databases can store data in flexible formats such as documents, key-value pairs, graphs and columns.

MongoDB is a popular document-oriented NoSQL database.

It stores information in BSON documents, which are similar to JSON objects.

MongoDB structure



Important terms

MongoDB	Meaning
Database	Container for collections
Collection	Group of documents
Document	Individual record
Field	Key-value pair
`_id`	Unique document identifier

Example document:

```
{
    name: "Ajay",
    department: "CSE",
    age: 20
}
```

SQL vs NoSQL

SQL	NoSQL
Relational database	Non-relational database model
Tables	Collections (in MongoDB)
Rows	Documents
Fixed schema commonly used	Flexible document structure
Uses SQL	Uses database-specific operations

25. Connecting Node.js with MongoDB

Module 4 – Core syllabus topic

Node.js can connect to MongoDB using the MongoDB Node.js driver.

The driver provides functions to communicate with the MongoDB server and perform database operations.

Requirements:

1. Node.js installed.
2. MongoDB server or MongoDB Atlas account.
3. MongoDB Node.js driver.
4. Database connection URL.

Step 1 – Create project

```bash
mkdir studentapp
cd studentapp

npm init -y
```

Step 2 – Install MongoDB driver

```bash
npm install mongodb
```

Step 3 – Import MongoDB

```javascript
const { MongoClient } = require('mongodb');
```

Step 4 – Create connection

```javascript
const uri = "mongodb://127.0.0.1:27017";

const client = new MongoClient(uri);
```

Step 5 – Connect to database

```javascript
await client.connect();

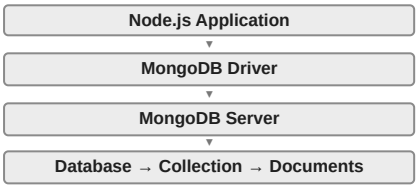
const db = client.db("college");

const students = db.collection("students");
```

Here:

- `MongoClient` creates a connection client.
- `connect()` connects to MongoDB.
- `db()` selects the database.
- `collection()` selects the collection.

Connection flow



26. CRUD Operations in MongoDB

CRUD stands for:

- Create
- Read
- Update
- Delete

These are the four basic operations used to manage database records.

Operation	MongoDB method	Purpose
Create	`insertOne()`	Insert document
Read	`find()` / `findOne()`	Retrieve documents
Update	`updateOne()`	Modify document
Delete	`deleteOne()`	Remove document

A. CREATE

Used to insert a new document into a collection.

```javascript
await students.insertOne({
    name: "Ajay",
    department: "CSE",
    age: 20
});
```

B. READ

Used to retrieve documents.

```javascript
const result = await students.find({}).toArray();

console.log(result);
```

To retrieve a particular student:

```javascript
const result = await students.findOne({
    name: "Ajay"
});

console.log(result);
```

C. UPDATE

Used to modify an existing document.

```javascript
await students.updateOne(
    { name: "Ajay" },
    { $set: { age: 21 } }
);
```

Here:

- First argument identifies the document.
- `$set` specifies the field to modify.

D. DELETE

Used to remove a document.

```javascript
await students.deleteOne({
    name: "Ajay"
});
```

Remember the four methods:

```
C  CREATE – insertOne()
R  READ   – find()
U  UPDATE – updateOne()
D  DELETE – deleteOne()
```

27. Main 15-mark PYQ program – Node.js + MongoDB CRUD

Must practise – Employee Management System

Your previous paper asks for a Node.js application to manage employee information stored in MongoDB.

The employee details include:

- Name
- Position
- Department
- Salary

The application should perform Create, Read, Update and Delete operations.

Program

First install:

```bash
npm install mongodb
```

Create `server.js`:

```javascript
const { MongoClient } = require('mongodb');

const uri = "mongodb://127.0.0.1:27017";

async function main() {

    const client = new MongoClient(uri);

    try {

        // 1. Connection
        await client.connect();

        console.log("MongoDB Connected");

        // 2. Database
        const db = client.db("company");

        // 3. Collection
        const employees = db.collection("employees");

        // CREATE
        await employees.insertOne({
            name: "Ajay",
            position: "Developer",
            department: "CSE",
            salary: 30000
        });

        console.log("Employee Added");

        // READ
        const result = await employees.find({}).toArray();

        console.log("Employee Records:", result);

        // UPDATE
        await employees.updateOne(
            { name: "Ajay" },
            { $set: { salary: 40000 } }
        );

        console.log("Employee Updated");

        // DELETE
        await employees.deleteOne({
            name: "Ajay"
        });

        console.log("Employee Deleted");

    }

    catch (error) {
        console.log(error);
    }

    finally {
        await client.close();
        console.log("Connection Closed");
    }
}

main();
```

Execution

```bash
node server.js
```

Expected console output:

```
MongoDB Connected
Employee Added
Employee Records: [...]
Employee Updated
Employee Deleted
Connection Closed
```

Program explanation – What to write in the exam

1. Import the MongoDB driver using `require()`.
2. Create a MongoClient object using the database connection URL.
3. Establish a connection using `connect()`.

4. Select the database using `db()`.
5. Select the employee collection using `collection()`.
6. Insert employee details using `insertOne()`.
7. Retrieve employee records using `find()`.
8. Modify salary using `updateOne()` and `$set`.
9. Remove the employee using `deleteOne()`.
10. Close the database connection using `close()`.

How to adapt this program to other PYQs

If question asks for	Change the collection and fields
Student management	`students`, name, registerNo, department, marks
Book management	`books`, title, author, price
Product management	`products`, name, price, quantity
Employee management	`employees`, name, position, department, salary

The connection and CRUD structure remain the same.

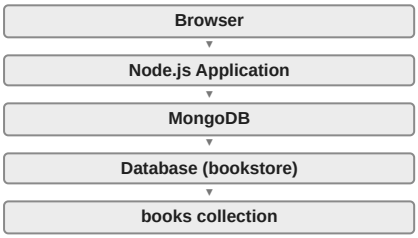
Important: This is a sequential demonstration of all four CRUD operations. A real application would normally expose separate API endpoints for each operation rather than inserting and deleting the same record during every server start.

MODULE 4 — NODE.JS + MONGODB CRUD (10-MARK PROGRAM, from scanned notes)

Online Bookstore — CRUD Using Node.js and MongoDB

This is the highest-priority 10-mark question. CRUD means **C**reate, **R**ead, **U**pdate, **D**elete. For a bookstore: CREATE = add book, READ = display books, UPDATE = change book, DELETE = remove book.

Basic architecture



Required package

```bash
npm install mongodb
```

Master Node + MongoDB CRUD program

```javascript
const { MongoClient } = require("mongodb");

const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);

async function main() {
    try {
        // CONNECT
        await client.connect();
        console.log("MongoDB Connected");

        const db = client.db("bookstore");
        const books = db.collection("books");

        // ========================
        // CREATE
        // ========================
        await books.insertOne({
            title: "JavaScript Basics",
            author: "John",
            genre: "Programming",
            stock: 10
        });
        console.log("Book inserted");

        // ========================
        // READ
        // ========================
        const allBooks = await books.find().toArray();
        console.log("Books:");
        console.log(allBooks);

        // ========================
        // UPDATE
        // ========================
        await books.updateOne(
            { title: "JavaScript Basics" },
            { $set: { stock: 15 } }
        );
        console.log("Book updated");

        // ========================
        // DELETE
        // ========================
        await books.deleteOne({
            title: "JavaScript Basics"
        });
        console.log("Book deleted");

    } catch (error) {
        console.log(error);
    } finally {
        await client.close();
        console.log("MongoDB connection closed");
    }
}

main();
```

Output (Command Prompt – node app.js):

```plain text
MongoDB Connected
Book inserted
Books:
[
  {
    _id: new ObjectId("665f1c2a9b1e8a3d4c7e5a10"),
    title: 'JavaScript Basics',
    author: 'John',
    genre: 'Programming',
    stock: 10
  }
]
Book updated
Book deleted
MongoDB connection closed
```

Understand this code instead of memorizing blindly

Connection — connects Node.js to MongoDB:

```javascript
const { MongoClient } = require("mongodb");
const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);
await client.connect();
```

Database — creates/accesses the `bookstore` database:

```javascript
const db = client.db("bookstore");
```

Collection — the collection is `books`:

```javascript
const books = db.collection("books");
```

CREATE — adds a document:

```javascript
await books.insertOne({
    title: "JavaScript Basics",
    author: "John",
    genre: "Programming",
    stock: 10
});
```

READ — retrieves documents:

```javascript
const allBooks = await books.find().toArray();
console.log(allBooks);
```

UPDATE — finds the book and changes `stock: 10` to `stock: 15`:

```javascript
await books.updateOne(
    { title: "JavaScript Basics" },
    { $set: { stock: 15 } }
);
```

DELETE — deletes the matching document:

```javascript
await books.deleteOne({
    title: "JavaScript Basics"
});
```

What to write in the 10-mark answer

1. **Definition:** CRUD represents Create, Read, Update and Delete operations used to manage data in a database.
2. **Connection architecture:** Node.js → MongoDB → Database → Collection
3. **Installation:** `npm install mongodb`
4. **Connection code**
5. **Create**
6. **Read**
7. **Update**
8. **Delete**
9. **Output**

How to adapt this to different questions

Question becomes	Collection and fields
Online bookstore (previous paper)	`books`: title, author, genre, stock
Employee management system	`employees`: name, department, salary
Product management	`products`: name, price, category, stock
Student management	`students`: name, department, mark

The CRUD structure doesn't change.

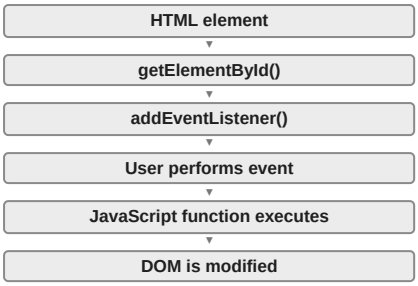
CLIENT-SIDE SUPPLEMENT — DOM, ANGULARJS, REST API, SPA (from scanned notes)

1. DOM Event Handling Using JavaScript (10-mark program)

What you need to know

- DOM = Document Object Model
- HTML elements become DOM objects
- JavaScript can access and modify them
- Events = user/browser actions
- Event handling = responding to events
- `getElementById()`, `addEventListener()`, `.value`, `.innerHTML`, `.style`

The master pattern is:



Master DOM program

Demonstrates: button click, input, DOM access, changing text, changing CSS, calculation, multiple event handlers.

```HTML
<!DOCTYPE html>
<html>
<head>
    <title>DOM Event Handling</title>
    <style>
        body {
            font-family: Arial;
            padding: 30px;
        }
        .box {
            width: 400px;
            padding: 20px;
            border: 2px solid black;
        }
        button {
            padding: 10px;
            margin: 5px;
        }
    </style>
</head>

<body>

<div class="box">
    <h2 id="title">Product Calculator</h2>

    <label>Product Name:</label>
    <input type="text" id="name">
    <br><br>

    <label>Price:</label>
    <input type="number" id="price">
    <br><br>

    <label>Quantity:</label>
    <input type="number" id="quantity">
    <br><br>

    <button id="calculate">Calculate</button>
    <button id="change">Change Style</button>

    <h3 id="result"></h3>
</div>

<script>
document.getElementById("calculate").addEventListener("click",
function() {

    let name = document.getElementById("name").value;
    let price = document.getElementById("price").value;
    let quantity = document.getElementById("quantity").value;

    let total = price * quantity;

    document.getElementById("result").innerHTML =
        "Product: " + name + "<br>Total Price: ₹" + total;
});

document.getElementById("change").addEventListener("click", function()
{

    document.getElementById("title").style.color = "blue";
    document.getElementById("result").style.fontSize = "20px";
});
</script>

</body>
</html>
```

Output: Product Name = Pen, Price = 50, Quantity = 3 → **Product: Pen / Total Price: ₹150**.

How to adapt this to ANY question

Question: *Develop a student result application using JavaScript DOM event handling.* Change Product Name → Student Name, Price → Mark, Quantity → Subject, Total → Average/Total Mark. The event-handling structure remains exactly the same.

Question: *Create a ticket booking system.* Change Product → Movie, Price → Ticket Price, Quantity → Number of Tickets, Total → Ticket Cost.

What to write for theory

DOM: The Document Object Model (DOM) represents an HTML document as a tree of objects. JavaScript can access and manipulate these objects to dynamically change HTML content, attributes and CSS styles.

Event: An event is an action that occurs in a web page, such as a mouse click, keyboard input or form submission.

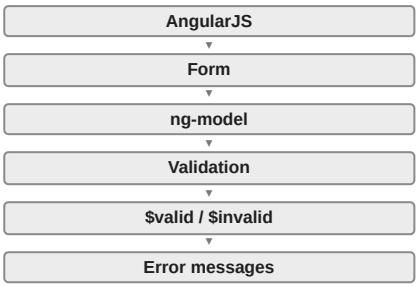
Event handling: Event handling is the process of detecting an event and executing a function in response to that event.

```JAVASCRIPT
element.addEventListener("click", function() {
    // action
});
```

`addEventListener()` attaches an event handler to an HTML element.

2. Form Validation Using AngularJS (10-mark program)

The examiner can turn it into: *Develop an AngularJS application for validating a registration form* (or employee/student/customer details). The actual application doesn't matter much. The important part is:



AngularJS validation concepts you absolutely need

Concept	Meaning
`ng-app`	Starts AngularJS
`ng-controller`	Connects the controller
`ng-model`	Connects input to data
`required`	Makes field mandatory
`ng-minlength`	Minimum characters
`ng-maxlength`	Maximum characters
`$valid`	Form/input is valid
`$invalid`	Form/input is invalid
`$touched`	User has interacted with the field

Master AngularJS form validation program

```HTML
<!DOCTYPE html>
<html>
<head>
    <title>AngularJS Form Validation</title>

    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min
.js">
    </script>

    <style>
        body {
            font-family: Arial;
            padding: 30px;
        }
        .box {
            width: 400px;
            padding: 20px;
            border: 2px solid black;
        }
        input {
            width: 90%;
            padding: 8px;
            margin: 5px;
        }
        .error {
            color: red;
        }
        button {
            padding: 10px 20px;
        }
    </style>
</head>

<body ng-app="myApp">

<div ng-controller="formController">
    <div class="box">

        <h2>Registration Form</h2>

        <form name="myForm" ng-submit="submitForm()" novalidate>

            <!-- NAME -->
            <label>Name:</label>
            <input
                type="text"
                name="name"
                ng-model="user.name"
                required
                ng-minlength="3">

            <p class="error"
               ng-show="myForm.name.$touched && myForm.name.$invalid">
                Name must contain at least 3 characters.
            </p>

            <!-- EMAIL -->
            <label>Email:</label>
            <input
                type="email"
                name="email"
```

```
                ng-model="user.email"
                required>

            <p class="error"
               ng-show="myForm.email.$touched &&
myForm.email.$invalid">
                Enter a valid email address.
            </p>

            <!-- AGE -->
            <label>Age:</label>
            <input
                type="number"
                name="age"
                ng-model="user.age"
                required
                min="18"
                max="60">

            <p class="error"
               ng-show="myForm.age.$touched && myForm.age.$invalid">
                Age must be between 18 and 60.
            </p>

            <!-- PASSWORD -->
            <label>Password:</label>
            <input
                type="password"
                name="password"
                ng-model="user.password"
                required
                ng-minlength="6">

            <p class="error"
               ng-show="myForm.password.$touched &&
                        myForm.password.$invalid">
                Password must contain at least 6 characters.
            </p>

            <br>

            <button type="submit"
                    ng-disabled="myForm.$invalid">
                Submit
            </button>

        </form>

        <h3>{{message}}</h3>

    </div>
</div>

<script>
var app = angular.module("myApp", []);

app.controller("formController", function($scope) {

    $scope.user = {};
    $scope.message = "";

    $scope.submitForm = function() {

        if ($scope.myForm.$valid) {
            $scope.message =
                "Form submitted successfully!";
        }
    };
});
</script>

</body>
</html>
```

Output: invalid input (e.g. "Aj", "ajay@", age 17, short password) shows red error messages and keeps **Submit** disabled; valid input (Ajay, ajay@gmail.com, 21, password) enables Submit and shows **Form submitted successfully!**

Understand the important pieces

AngularJS starts here:

```
<body ng-app="myApp">
```
HTML

Controller:

```
<div ng-controller="formController">
```
HTML

Form — AngularJS gives the form an object `myForm`:

```
<form name="myForm">
```
HTML

Input binding — `ng-model` connects the input with `user.name`:

```
<input ng-model="user.name" required>
```
HTML

Validation:

- `required` means the field cannot be empty.
- `ng-minlength="3"` means minimum 3 characters.
- `type="email"` makes AngularJS validate the input as an email.

`$invalid` — `myForm.name.$invalid` means the name field is currently invalid.

`$valid` — `myForm.$valid` means the entire form is valid.

`$touched` — `myForm.name.$touched` means the user has interacted with the field. So:

```
ng-show="myForm.name.$touched && myForm.name.$invalid"
```
HTML

means: show the error only after the user has interacted with the field and the value is invalid.

How to adapt this to any question

Change Name / Email / Age / Password to whatever fields the question asks. Employee registration → Employee Name, Email, Salary, Department. Product form → Product Name, Price, Quantity, Category. The validation mechanism remains the same.

3. AngularJS Directives

An AngularJS directive is a special instruction added to HTML that extends HTML's functionality. Directives usually begin with `ng-`.

Directive	Purpose
`ng-app`	Defines AngularJS application
`ng-controller`	Specifies controller
`ng-model`	Provides data binding
`ng-init`	Initializes data
`ng-repeat`	Repeats HTML elements
`ng-click`	Handles click
`ng-show`	Shows element
`ng-hide`	Hides element
`ng-bind`	Binds/displays data

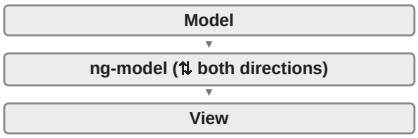
Example:

```
<div ng-app="myApp" ng-controller="myCtrl">
    <input ng-model="name">
    <p>{{name}}</p>
</div>
```
HTML

2-mark answer: AngularJS directives are special instructions added to HTML that extend its functionality and provide AngularJS features. Examples include `ng-app`, `ng-controller`, `ng-model`, `ng-repeat` and `ng-click`.

Which directive binds HTML controls to data? → `ng-model`. Which directive initializes AngularJS? → `ng-app`.

4. Two-Way Data Binding in AngularJS



Data flows in both directions.

```
<input ng-model="name">
<p>Hello {{name}}</p>
```
HTML

If the model changes, the view updates:

```
$scope.name = "Ajay";
```
JAVASCRIPT

If the user types something into the input (Input → Model), the model also updates.

2-mark answer: Two-way data binding in AngularJS synchronizes the model and the view. Changes in the model are automatically reflected in the view, and changes made by the user in the view update the model. The `ng-model` directive is used for two-way binding.

Output: typing "Ajay" in the input instantly shows *Hello Ajay* (Model ↔ View). Most important keyword: `ng-model`.

5. Filters in AngularJS

A filter is used to format or transform data before displaying it.

```
{{name | uppercase}}
```
HTML

If `name = "ajay"` the output is `AJAY`.

Filter	Purpose
`uppercase`	Converts to uppercase
`lowercase`	Converts to lowercase
`currency`	Formats currency
`number`	Formats numbers
`date`	Formats dates
`orderBy`	Sorts data
`filter`	Filters data
`limitTo`	Limits displayed data

Examples:

```
{{name | uppercase}}
{{price | currency}}
{{name | lowercase}}
{{students | orderBy:'name'}}
```
HTML

2-mark answer: Filters in AngularJS are used to format or transform data before displaying it in the view. Common built-in filters include `uppercase`, `lowercase`, `currency`, `number`, `date`, `orderBy`, `filter` and `limitTo`.

6. Modules and Controllers in AngularJS

Module: A module is a container for an AngularJS application and its components.

```javascript
var app = angular.module("myApp", []);
```

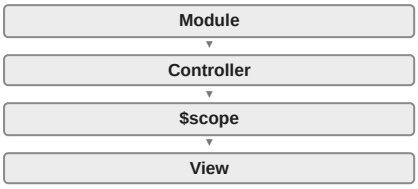
Controller: A controller contains the application logic and data.

```javascript
app.controller("myCtrl", function($scope) {
    $scope.name = "Ajay";
});
```

HTML:

```html
<div ng-app="myApp" ng-controller="myCtrl">
    {{name}}
</div>
```

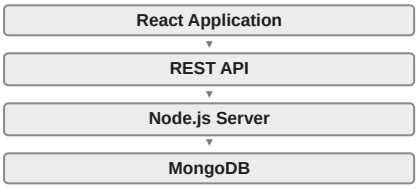
Output: `Ajay`



2-mark answer: A module is a container that organizes an AngularJS application and its components. A controller contains the application's logic and data and communicates with the view using `$scope`.

7. Role of REST API

REST = Representational State Transfer. A REST API allows different applications to communicate over HTTP.



Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Delete data

Example: `GET /students` gets students; `POST /students` creates a student.

2-mark answer: A REST API enables communication between client and server using HTTP. It allows resources to be created, retrieved, updated and deleted using methods such as POST, GET, PUT and DELETE. REST APIs are commonly used to connect web applications with server-side applications and databases.

Remember: GET → Read, POST → Create, PUT → Update, DELETE → Delete. This is directly connected to the Node.js + MongoDB CRUD question.

8. Traditional Website vs Single Page Application

Traditional website: when moving between pages, the browser generally requests and loads another HTML page (Page 1 → Reload → Page 2 → Reload → Page 3).

SPA (Single Page Application): the application loads a main page and JavaScript dynamically updates the content without requiring a complete page reload for every action (One main page → JavaScript → Content changes dynamically).

Traditional Website	SPA
Multiple page loads	Mainly one page
Full-page reloads	Dynamic updates
Navigation can involve server requests for new pages	JavaScript updates the view
Simpler traditional architecture	Rich interactive UI

2-mark answer: A traditional website loads separate HTML pages and may perform a full-page reload during navigation. A Single Page Application (SPA) loads one main page and dynamically updates its content using JavaScript without completely reloading the page. Examples of SPA technologies: AngularJS, React, Vue.

9. Last-Minute Memory Sheet

Topic	Remember
NPM	`node -v` · `npm -v` · `npm init -y` · `npm install package`
REST API	GET → Read · POST → Create · PUT → Update · DELETE → Delete
Traditional vs SPA	multiple pages + reload vs one main page + dynamic update
AngularJS directives	`ng-app` `ng-controller` `ng-model` `ng-init` `ng-repeat` `ng-click` `ng-show` `ng-hide` `ng-bind`
React Hello World	Component → JSX → `createRoot()` → `render()`
Two-way binding	Model ⇄ View (`ng-model`)
AngularJS filters	`uppercase` `lowercase` `currency` `number` `date` `orderBy` `filter` `limitTo`
Module + Controller	Module → Controller → `$scope` → View
Node.js file operations	`fs.writeFile()` `fs.readFile()` `fs.appendFile()` `fs.unlink()`